

Data Base Management System

Lecture - 7

Topics →

- * Decomposition

 - ↳ Lossy
 - ↳ Lossless

- * Normalization Theory

 - ↳ Functional Dependency
 - ↳ Multivalued Dependency

- * Dependency Preservation

- * BCNF

- * Normal Forms

- * Canonical Cover

Functional Dependency

Functional dependency is relationship between two attributes in a relation where one attribute uniquely determines the value of another attribute

FD - $X \rightarrow Y$

For each value of X there is one corresponding value of Y

X is called determinant

Y is called dependant

► Properties of Functional Dependencies:

1. Reflexivity:

If $Y \subseteq X$, then $X \rightarrow Y$.

This means that if you already know a set of attributes X , you trivially know any subset Y of X .

Example: $(StudentID, Name) \rightarrow Name$ (since $Name$ is part of the left-hand side).

2. Augmentation:

If $X \rightarrow Y$, then for any set of attributes Z , we have $XZ \rightarrow YZ$.

Adding attributes to both sides of a functional dependency does not change its validity.

Example: If $StudentID \rightarrow Name$, then $(StudentID, Department) \rightarrow (Name, Department)$.

3. Transitivity:

If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.

Example: If $StudentID \rightarrow Department$ and $Department \rightarrow Faculty$, then $StudentID \rightarrow Faculty$.

4. Union:

If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$.

Combining two functional dependencies with the same determinant results in a new functional dependency.

5. Decomposition:

If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$.

If one functional dependency involves a combination of attributes on the right side, it can be split into separate dependencies.



🔥 Types of Functional Dependencies:

1. Trivial Functional Dependency:

A functional dependency $X \rightarrow Y$ is **trivial** if Y is a subset of X .

Example: $(StudentID, Name) \rightarrow Name$ is trivial because **Name** is already part of the left-hand side.

2. Non-Trivial Functional Dependency:

A functional dependency $X \rightarrow Y$ is **non-trivial** if Y is **not** a subset of X .

3. Fully Functionally Dependent:

If an attribute is functionally dependent on the entire primary key and not just part of it, it's **fully functionally dependent**. This is especially important in **2NF**.

4. Partial Functional Dependency:

A dependency where an attribute depends on only a **part** of the primary key (relevant in **2NF**). It is eliminated during normalization.

5. Transitive Dependency:

A dependency where one attribute is dependent on another via a third attribute (relevant in **3NF**).

Canonical Cover

Canonical Cover also known as Minimal Cover is a minimal set of functional dependency which has following characteristics :-

- * No extraneous attributes
- * No redundant functional dependencies

Steps of finding canonical cover

- * Remove Extraneous Attribute
- * Remove Redundant functional dependency
- * Decompose complex functional dependency

→ $x \rightarrow Y$ - x is EA so $F - \{x \rightarrow Y\}$ implies $x - \{A\} \rightarrow Y$ A is an attribute of x, remove A from x

→ $x \rightarrow Y$ and x is redundant

$F - \{x \rightarrow Y\}$ implies $x \rightarrow Y$

3) $X \rightarrow YZ$ becomes

$X \rightarrow Y$ and $X \rightarrow Z$

Example - $R(A, B, C, D)$

$F = \{A \rightarrow BC, AB \rightarrow D, C \rightarrow A, A \rightarrow B\}$

Step - 1}

* $A \rightarrow BC$ to $A \rightarrow B$ and $A \rightarrow C$ as

$A \rightarrow B$ already exists remove $A \rightarrow BC$

and add $A \rightarrow C$

* $AB \rightarrow D$ to $A \rightarrow D$ as B is EA

Step - 2}

as $A \rightarrow B$ and $C \rightarrow A$ implies $A \rightarrow C$

remove $A \rightarrow C$

Step - 3} $A \rightarrow BC$ becomes $A \rightarrow C$ &

$A \rightarrow B$

So, Canonical cover, $F_c = \{A \rightarrow B, A \rightarrow C, AB \rightarrow D, C \rightarrow A\}$

Normalization

* It removes duplicacy and redundancy

SID	S name	Age
1	Rahat	20
2	Razi	21
1	Rahat	20

Row level duplicacy

This can be removed by using primary key

Primary Key → Unique + Not null

Then comes Column Level Duplicacy

SID	S name	Cid	Cname	F id	F name	Salary
1	Ratul	C ₁	DBMS	F ₁	John	3000
2	Radi	C ₂	JAVA	F ₂	Raida	2
3	Rajesh	C ₃	DBMS	F ₁	John	3000
4	Chishti	C ₄	JAVA	F ₂	Raida	2
5	Zawad	C ₅	OOP	F ₃	JIM	100K

What happens when we have column Level Duplication

* Insertion Anomaly

* Deletion Anomaly

* Updation Anomaly

* If we try to add course in the following table, we must add a student id in it too as it is the primary key. We cannot insert dummy data which will create problem later on.

* Deletion will create problem like if we delete a course it will delete students also. If we delete students it will delete courses too.

* If we update data of a row, like if we update Chishti's name to A2SMG Musabbheen Chishti, it will change but no problem will occur. But if we update Raida maams salary to 50k, it will only be applicable for 1 row. Other rows will stay the same.

but if we run query to change all the salaries where faculty name is Raida, The update query will run many times checking each row

Now what normalization does?

we will divide the tables like

S id , S name

C id, C name

F id	F name	Salary
------	--------	--------

and we will have junction tables to complete the relationship between the tables

Basically speaking normalization of table makes data

- * Easier to understand
- * Easier to enhance and extend

* Protects from

→ Insertion anomalies

→ Update anomalies

→ deletion anomalies

First Normal Form

* Violation of 1NF (1)

→ Using row order to convey into



Fix → Adding Column Data

Suppose we want to order members
of سہیل گروپ سٹب tallest to shortest

Taseen
Riyad
Rafi
Chishti
Ahiz

here it is now order to convey the information

The correct way is →

Name	Height
Taseen	6 ft
Ahiz	5 ft 8 5 inch

* Violation of 1NF (2)

mixing data types in a column

* Violation of 1NF (3)

having column with no primary key

* Violation of 1NF (4)

Storing a repeating group of data on a single row



Designing table wisely / taking a new row for each data

Player ID	Quant 1	I-T-1	Quant 2	I-T 2	Quan-3	I-T.3
Jdog 21	2	amulets	4	rings		
gila 19	18	copper-coin				
Trev 73	3	shield	5	arrow	300	c-coin

This table is wrong, fix is in another page

Player ID	Item_Type	Item_Quantity
Jdog21	amulets	2
Jdog21	rings	4
gila9	copper coin	18
trav73	shields	3
trav73	arrows	5
trav73	copper coin	30
trav73	rings	7

Rules

- 1) Using row order to convey information is not permitted
- 2) Mixing data types is not permitted
- 3) Having tables without primary key is not permitted
- 4) Repeating groups is not permitted

Second Normal Form

Player_Inventory

 Player_ID	Item_Type	Item_Quantity	Player_Rating
jdog21	amulets	2	Intermediate
jdog21	rings	4	Intermediate
gilal9	copper coins	18	Beginner
trev73	shields	3	Advanced
trev73	arrows	5	Advanced
trev73	copper coins	30	Advanced
trev73	rings	7	Advanced

here, this is a bad design, As
if gilal9 spend all his copper
coin, now If we run a query to
find player rating of gilal9 we
will find null value

This is a deletion anomaly.

Player_Inventory

 Player_ID	Item_Type	Item_Quantity	Player_Rating
jdog21	amulets	2	Intermediate
jdog21	rings	4	Intermediate
gilal9	copper coins	18	Beginner
trev73	shields	3	Advanced
trev73	arrows	5	Advanced
trev73	copper coins	30	Advanced
trev73	rings	7	Advanced

if we update player rating in

1 column then there will be same player with 2 player rating

This is update anomaly

Again happens insertion anomaly if we push data which has 1 null column

* 2NF is about a table's non-key column relate to the primary key

→ Each non key attribute must depend on the entire primary key

if A, B forms a composite key and C is a non key attribute depends

Only on A So this partial dependency

2NF allows, $(A, B) \rightarrow C$ but not with partial dependency

Third Normal Form



Player

 Player_ID	Player_Rating	Player_Skill_Level
jdog21	Intermediate	4
gilal9	Beginner	3
trev73	Advanced	8
tina42	Beginner	1

if we update player skill level and failed to update player rating
This will cause data inconsistency

here,

Player_ID → Player_Skill_Level

Player_ID → Player_Skill_Level
Player_Rating ←

This type of dependency is called transitive dependency. 3NF form forbids this kind of dependency.

* Transitive dependency →

When a non key attribute is dependent on a candidate key attribute through another non key attribute.

BCNF → Every attribute on the table should depend on the key, whole key and nothing but the key.

3NF → Every non key attribute on the table should depend on the key, whole key and nothing but the key.

Boyce - Codd Normal Form

A higher version of 3NF

Rule $\rightarrow$

For every functional dependency
 $X \rightarrow Y$, X is a super key

* X can uniquely identify a row
in a table

* X can be a candidate key or
Composite key

Fourth - Normal Form



DesignMyBirdhouse.com



Model "Tweety"

Available colors:
Yellow, Blue

Available styles:
Bungalow, Duplex



Model "Metro"

Available colors:
Brown, Grey

Available styles:
High-Rise, Modular



Model "Prairie"

Available colors:
Brown, Beige

Available styles:
Bungalow, Schoolhouse

Model_Colours_And_Styles_Available

Model	Color	Style
Tweety	Yellow	Bungalow
Tweety	Yellow	Duplex
Tweety	Blue	Bungalow
Tweety	Blue	Duplex
Metro	Brown	High-Rise
Metro	Brown	Modular
Metro	Grey	High-Rise
Metro	Grey	Modular
Prairie	Brown	Bungalow
Prairie	Brown	Schoolhouse
Prairie	Beige	Bungalow
Prairie	Beige	Schoolhouse

This table follows 3NF but if we add a green Prairie model we need to make green prairie Bungalow and school house

This kind of dependency is called Multi varied dependency

{ Model } $\longrightarrow$ { color }

represented by $\longrightarrow$

$\{Model\} \longrightarrow \{style\}$

4NF $\rightarrow$ Multivalued dependencies in a table must be multivalued dependency on the key

Model_Colors_Available		Model_Styles_Available	
	Model		Model
	Color		Style
	Tweety		Bungalow
	Tweety		Duplex
	Metro		High-Rise
	Metro		Modular
	Prairie		Bungalow
	Prairie		Schoolhouse
	Prairie		
	Green		

Can be solved by decomposing the table

MVD $\rightarrow$ is when one attribute in a relationship determines multiple independent values of another attribute but without any direct relationship between those values

Fifth Normal Form

The table (which must be in 4NF) cannot be describable as the logical result of joining some other tables together

It is also known as Project Join Normal Form which mainly focuses on remove redundancies caused by join dependency

if a table is decomposed into multiple table and after being joined original table is found this will reduce join related complexity and dependencies
This the goal of 5NF

Decomposition

Decomposition is the process of breaking down complex database table into smaller related tables to reduce or remove redundancy and anomalies

There are Two types of decomposition

- * lossy
- * lossless

Lossy decomposition →

- * Data lost during decomposition
- * Original table cannot be reconstructed

Lossless decomposition →

- * No data lost

* Original table can be reconstructed

* Decomposition in 1NF

- Identify non atomic values
- Split repeating groups into separate rows or tables

* Decomposition in 2NF

- Identify partial dependencies
- Remove attributes that are partially dependent
- Create separate table.

$R(A, B, C, D)$

Functional dependency →

$(A, B) \rightarrow (C, D)$

$A \rightarrow C$

Fixed form →

$R_1 (A, C)$

$R_2 (B, D)$

$R_3 (A, B)$

* Decomposition in 3NF

→ Identity Transitive dependency

→ Remove non-key attribute that depends on another non-prime attribute

→ Create new table to store the transitive relations

Let be,

$A = \text{Student ID}$

$B = \text{Student Name}$

$C = \text{Dept Name}$

$D = \text{Dept head}$

$R (A, B, C, D)$

FDs :-

$A \rightarrow B, C$

$C \rightarrow D$

So, $A \rightarrow D$

After 3NF,

$R_1 (A, B, C)$

$R_2 (C, D)$

* Decomposition in BCNF

→ Identify overlapping candidate keys that causes redundancy

→ Decompose table to separate overlapping keys

Let be,

$A = \text{Course}$

$B = \text{Instructor}$

$C = \text{Room}$

$R(A, B, C)$

FDs -

$B \rightarrow C$

$A \rightarrow B$

After BCNF,

$R_1(A, B)$

$R_2(B, C)$

* Decomposition in 4NF

→ Identify Multivalued dependency

→ Check if the relation contains
any non-trivial MVDs

→ Decompose the relation to
eliminate MVDs

$R(S_ID, Skill, Hobby)$

$S_ID \twoheadrightarrow Skill$

$S_ID \twoheadrightarrow Hobby$

After decomposition

$R_1 (S_ID, Skill)$

$R_2 (S_ID, Hobby)$

$R_1 \bowtie R_2 \text{ on } S_ID = R$

* Decomposition in 5NF

→ Identify complex dependencies

→ Break the table into smaller table to eliminate join redundancy

$R (Actor, Movie, Role)$

Join Dependency -

$R \rightarrow (Actor, Movie), (Movie, Role),$
 $(Actor, Role)$

Join leads to redundancy & inconsistency

After Decomposition,

R1 (Actor, Movie)

R2 (Movie, Role)

R3 (Actor, Role)